

EECS 482

Lab 11: Filesystems

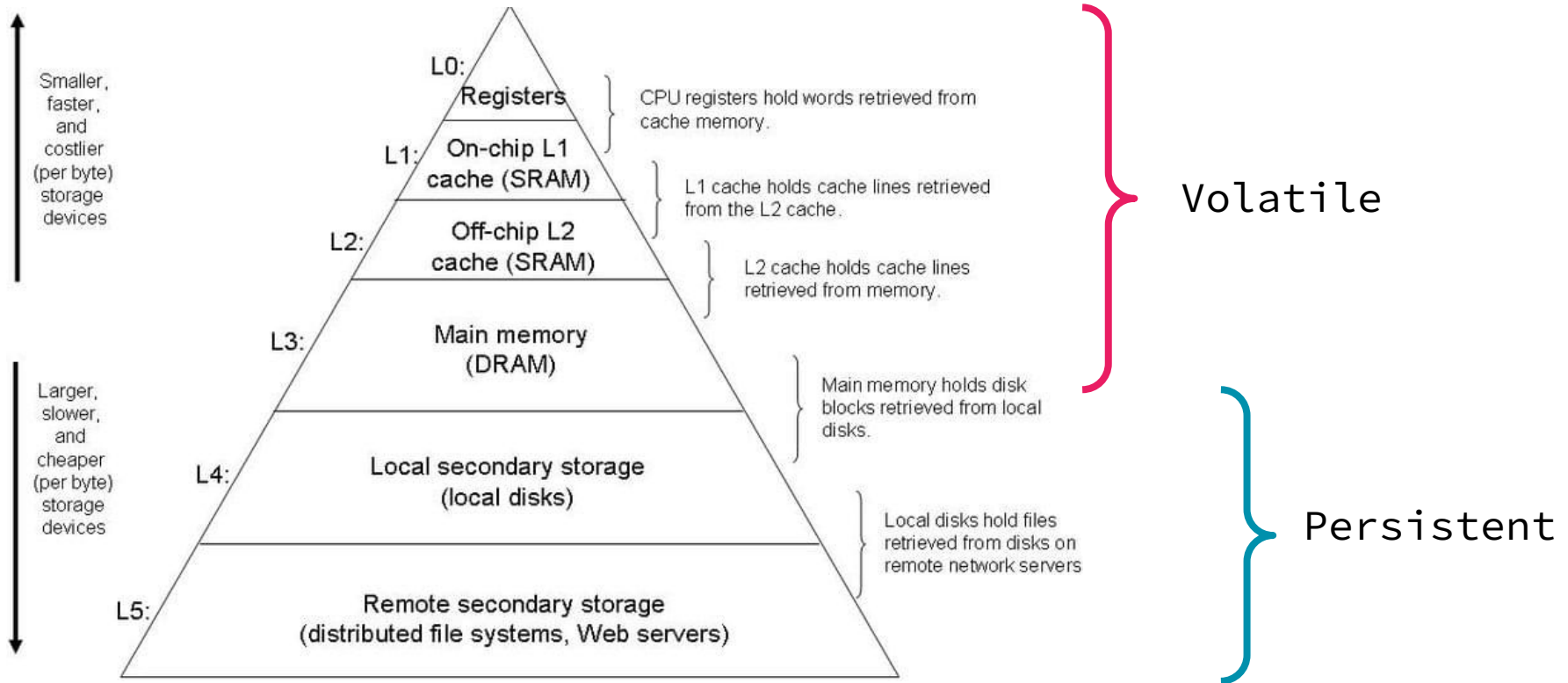
Agenda

1. File Systems
2. P4 Intro
3. On-disk structures
4. Installing Boost

— — —

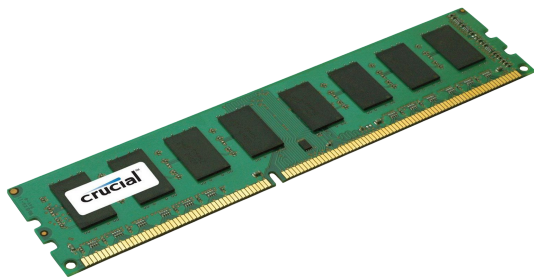
File Systems

The hierarchy of data



The hierarchy of data

DRAM



\$3.00 / GB

100 **nano**second
access latency

SSD



\$0.06 / GB

100 **micro**seconds
access latency

Hard Disk



\$0.02 / GB

1-10 **milli**second
access latency

Our end goal: provide a nice abstraction

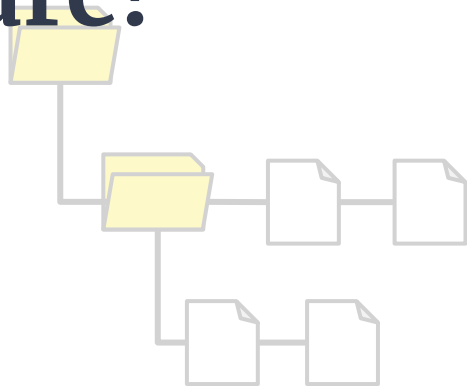
Hardware provides:

OS abstraction:

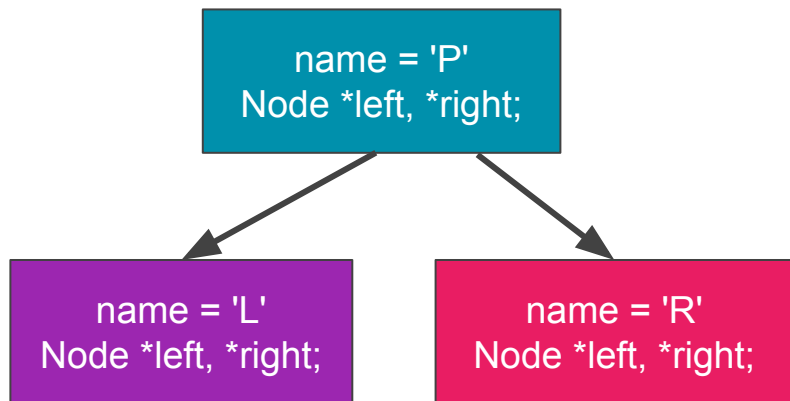
• Array of bytes
• contiguous blocks

How do we represent a hierarchy in a linear structure?

• Discrete files
• Hierarchical structure



Representing a hierarchy with linear structure

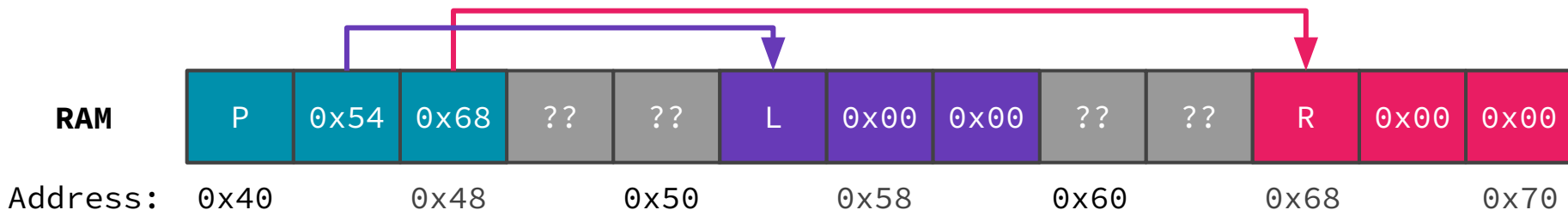


You've done this before!

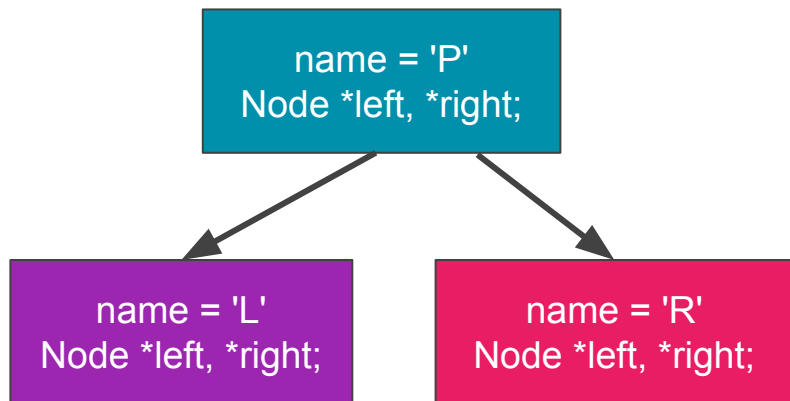
Hierarchy = EECS 280 tree

Linear structure = RAM

Pointers! (virtual addresses)



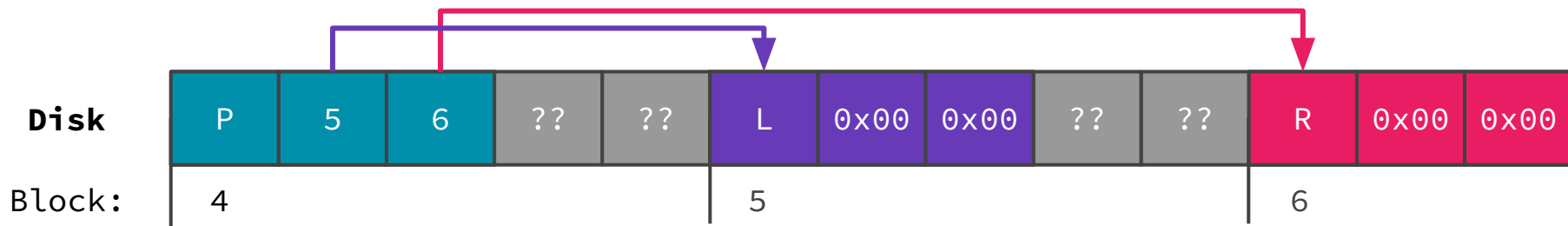
Representing a hierarchy with linear structure



On disk, it's the same idea.

"Pointers" point to disk blocks.

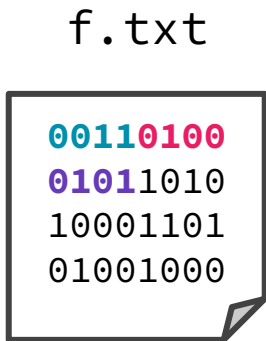
Pointers! (disk block numbers)



How to represent a file?

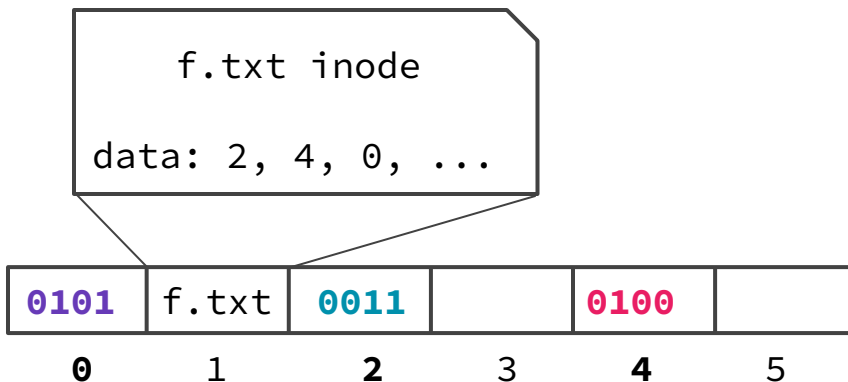
Abstraction:

- Named array of bytes



Implementation:

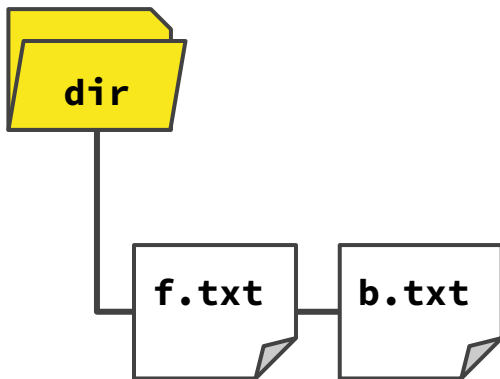
- Collection of disk blocks
- inode stores metadata



How to represent a directory?

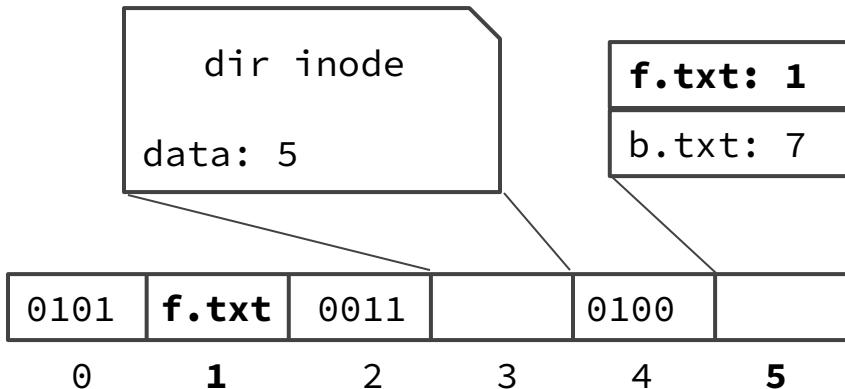
Abstraction:

- Named collection of files

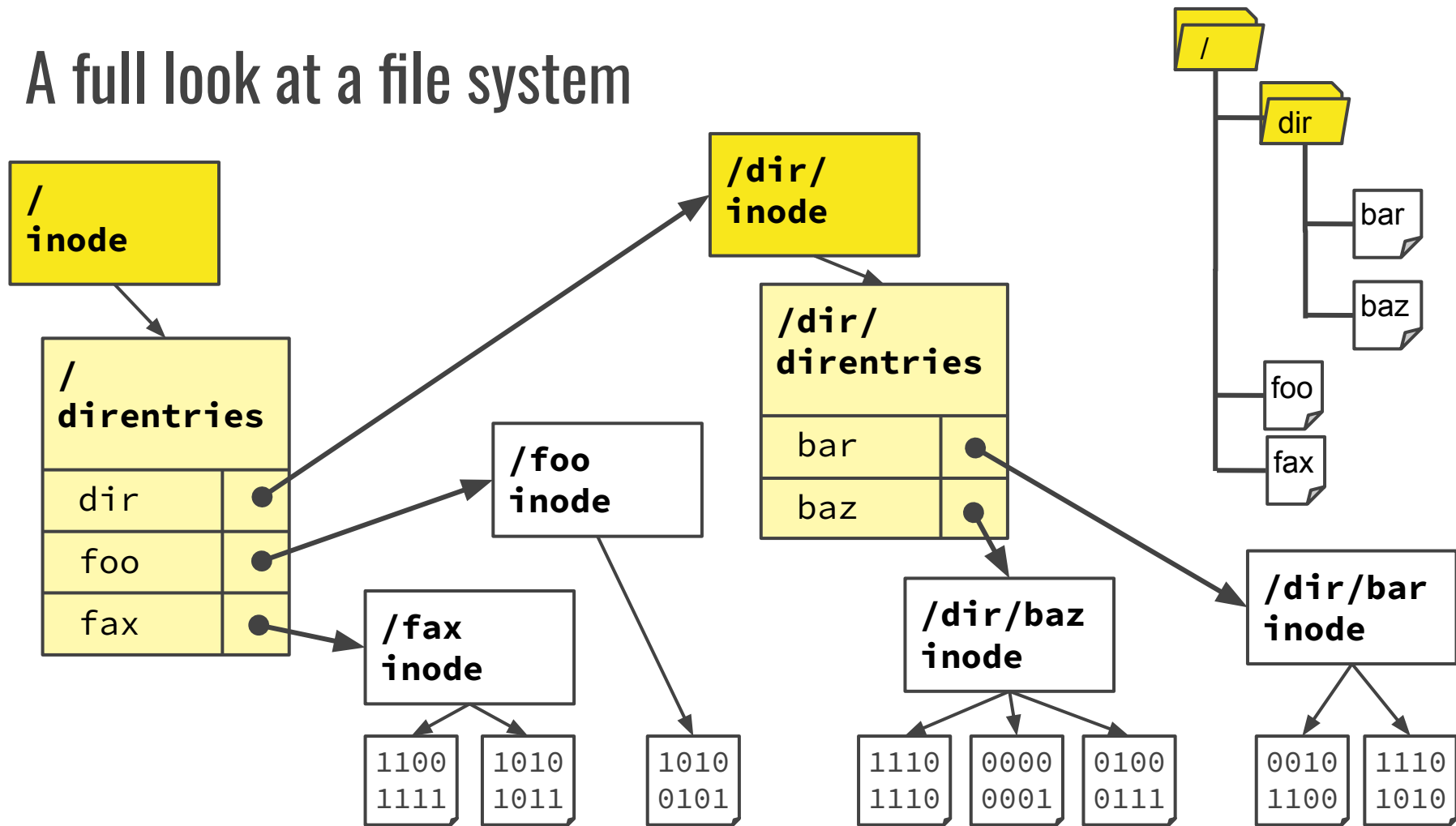


Implementation:

- Special file
- Map from name to inode



A full look at a file system



Getting Started with Project 4

Due 08/11! (~2.5 weeks)

What you have to implement

— — —

Client:

These are your **test** cases.

We provide you the RPC stubs:

- `fs_clientinit();`
- `fs_readblock();`
- ...

You will provide us:

- Test cases (need 17 bugs)
- Pre-existing FS (see section 9)

Server:

This is your **project**.

We provide you:

- `disk_readblock();`
- `disk_writeblock();`
- `print_port();`

You will provide us:

- `int main();`
- Server Networking + FS

P4 Suggestions

Network Setup and Test

Parsing Request and Validation

Single-Threaded Commands

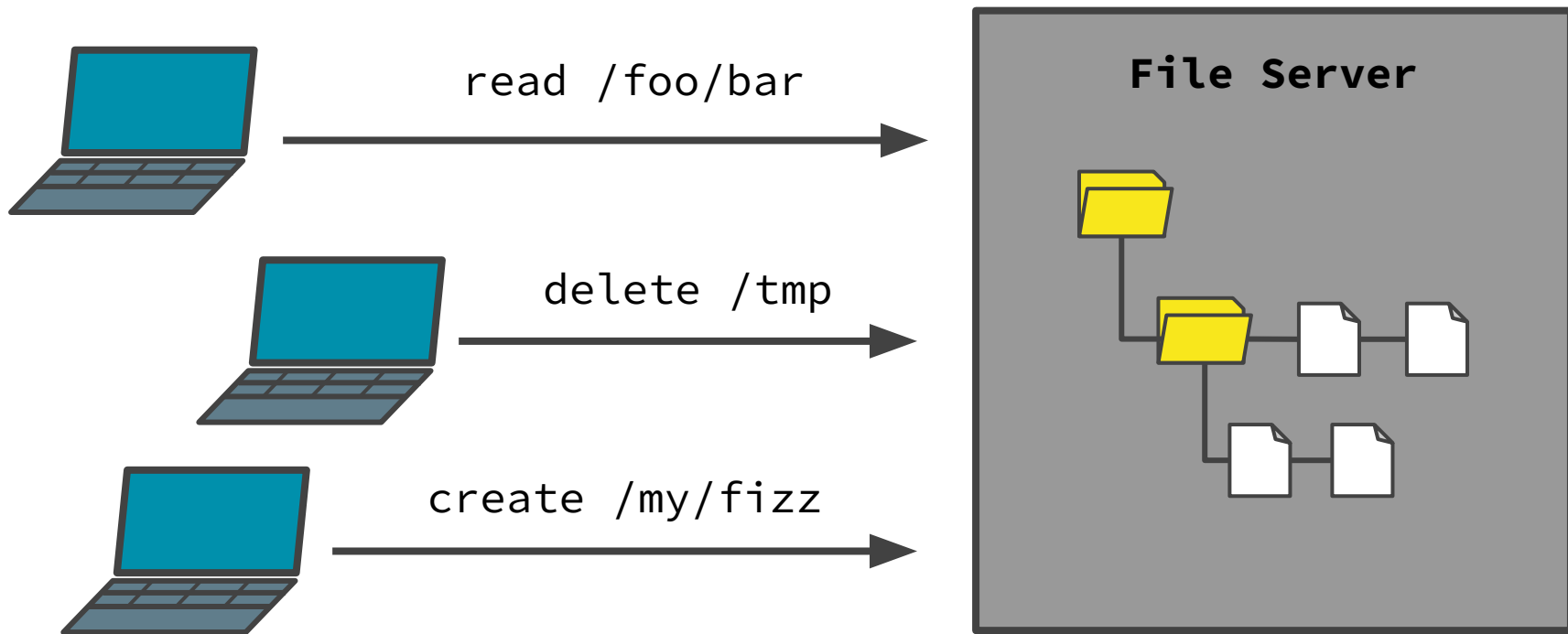
Multi-Threaded Commands

Using upgradable lock (6 crd) + Debugging/Testing



You should be here

Project 4 is Two Parts



Project 4 is Two Parts

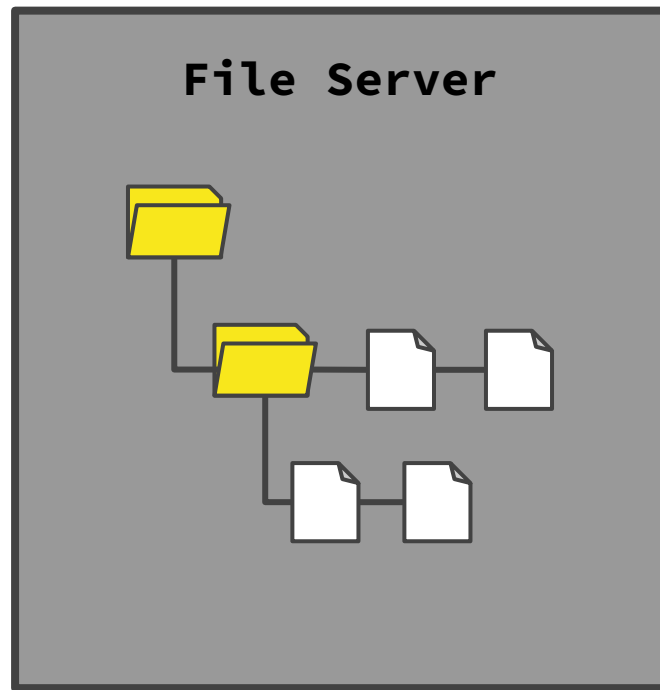
Parsing / Requests

socket code

parsing requests

validating requests

sending responses



Project 4 is Two Parts

Parsing / Requests

socket code

parsing requests

validating requests

sending responses

File System

disk_readblock/writeblock

fs tree traversal

validating requests

hand-over-hand locking

crash consistency

****Separate these two parts****

Parsing / Requests

socket code

parsing requests

validating requests

sending responses

File System

disk_readblock/writeblock

fs tree traversal

validating requests

hand-over-hand locking

crash consistency

****Separate these two parts****

Parsing / Requests

socket code

parsing requests

validating requests

start here

sending responses

File System

disk_readblock/writeblock

fs tree traversal

validating requests

hand-over-hand locking

crash consistency

Parsing

Parsing: the most important thing to remember

— — —

Know the difference between a C-string and a bunch of bytes.

Use the right function for the job.

C-strings

strlen()
strcpy()
std::string(buf)

Bucket-o-bytes

sizeof(*)
memcpy()
std::string(buf, len)

Other parsing tips

- The request header is null-terminated. The body is not.
 - *This changes how you do your `recv()` calls.*
- Make use of strings and stringstream.
 - *Don't worry about raw performance if it jeopardizes possible correctness or readability.*
- **Unit test** your parsing code to catch bugs.
 - *i.e., call your functions directly in some C++ program*
- **Note:** there is only one correct input format, but many incorrect
 - It is easier to check if something is correct than if it is incorrect

What is stringstream?

You can use `stringstream` to create datastreams that read from/write to strings in a fashion similar to `cin/cout`.

```
// istream denotes an input stringstream
std::istream iss("  Alice \t\t\t\t\t Bob \n\n\n  Carl  ???  :) ");
std::string str1, str2, str3;
iss >> str1 >> str2 >> str3;
std::cout << str3 << ' ' << str2 << ' ' << str1; // prints "Carl Bob Alice"
```

stringstream can help with p4!

Project 4 on-disk data structures

Project 4: inode structure (file)

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

```
type    = 'f'  
owner   = "username"  
size    = 2  
blocks  = [7, 2]
```

Block 7 (data)

we hold these truths
to be self-evident,

Block 2 (data)

that all men are
created equal...

fs_inode

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

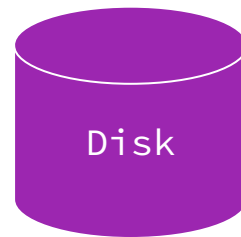
```
fs_inode inode;
```



fs_inode

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

```
fs_inode inode;  
disk_readblock(x, &inode);
```



f u s e r n a m e 0 0 0 0 0 0 2 0 0 0 7 0 0 0 2 ...

fs_inode

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

cout << inode.type

'f'

f u s e r n a m e 0 0 0 0 0 0 2 0 0 0 7 0 0 0 2 ...

fs_inode

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

```
cout << inode.owner
```

"username"

f u s e r n a m e 0 0 0 0 0 0 2 0 0 0 7 0 0 0 2 ...

fs_inode

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

```
cout << inode.size
```

"2"

f u s e r n a m e 0 0 0 0 0 2 0 0 0 7 0 0 0 2 ...

fs_inode

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

```
cout << inode.blocks[0]
```

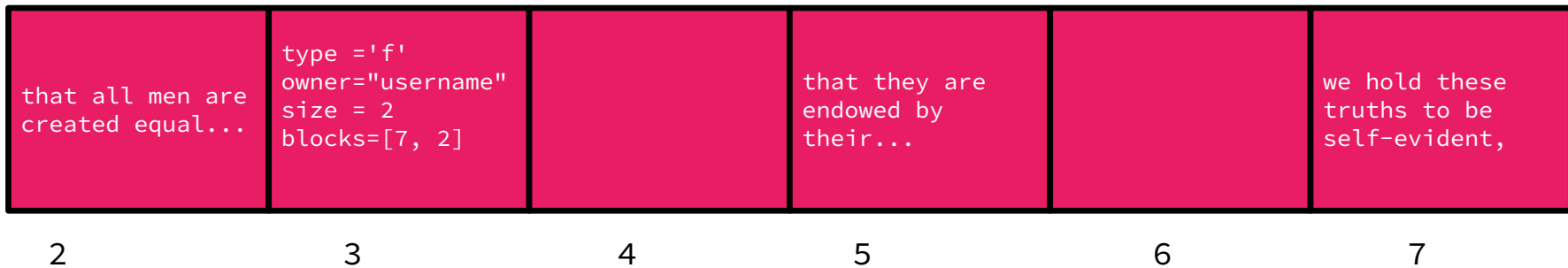
"7"

f u s e r n a m e 0 0 0 0 0 0 2 0 0 0 7 0 0 0 2 ...

fs_inode: adding a block

```
type    = 'f'  
owner   = "username"  
size    = 2  
blocks  = [7, 2]
```

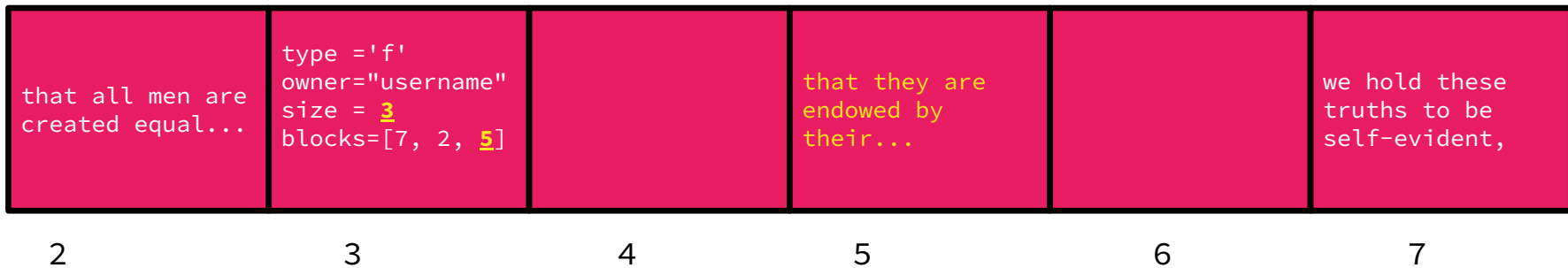
```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```



fs_inode: adding a block

```
type    = 'f'  
owner   = "username"  
size    = 3  
blocks  = [7, 2, 5]
```

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```



fs_inode: adding a block

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

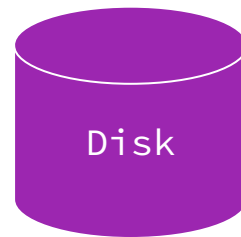
```
fs_inode inode;
```



fs_inode: adding a block

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

```
fs_inode inode;  
disk_readblock(x, &inode);
```

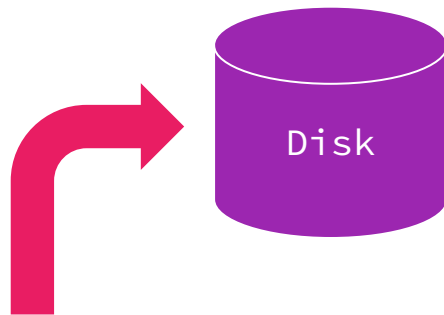


f u s e r n a m e 0 0 0 0 0 0 2 0 0 0 7 0 0 0 2 ...

fs_inode: adding a block

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

```
fs_inode inode;  
disk_readblock(x, &inode);  
...
```



f u s e r n a m e 0 0 0 0 0 0 3 0 0 0 7 0 0 0 2 0 0 0 5 ...

Project 4: inode structure (directory)

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

```
type    = 'd'  
owner   = "username"  
size    = 3  
blocks  = [13, 19, 2]
```



Block 13



Block 19



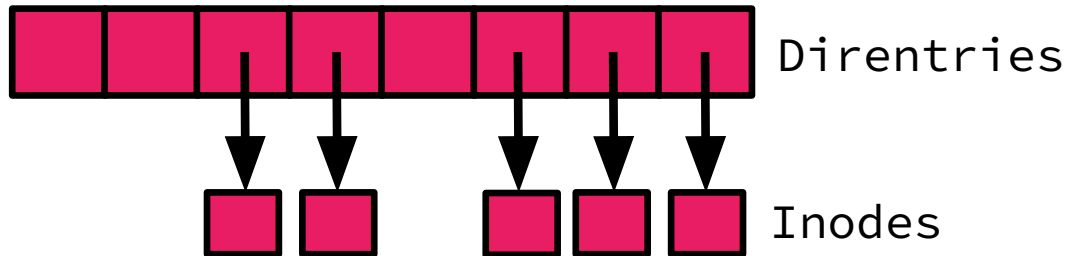
Block 2

Project 4: directory entries

```
struct fs_direntry {  
    char    name[FS_MAXFILENAME + 1];  
    uint32_t inode_block;  
};
```

name = "file1.txt"
inode_block = 4

**How many fs_direntries fit
in 1 disk block?**



Block 4
(inode)

type = 'f'
owner = "username"
size = 2
blocks = [7, 2]

Project 4: inode structure (directory)

```
struct fs_inode {  
    char    type;  
    char    owner[FS_MAXUSERNAME + 1];  
    uint32_t size;  
    uint32_t blocks[FS_MAXFILEBLOCKS];  
};
```

```
type    = 'd'  
owner   = "username"  
size    = 3  
blocks  = [13, 19, 2]
```



Block 13
(8x fs_dirent)



Block 19
(8x fs_dirent)



Block 2
(8x fs_dirent)

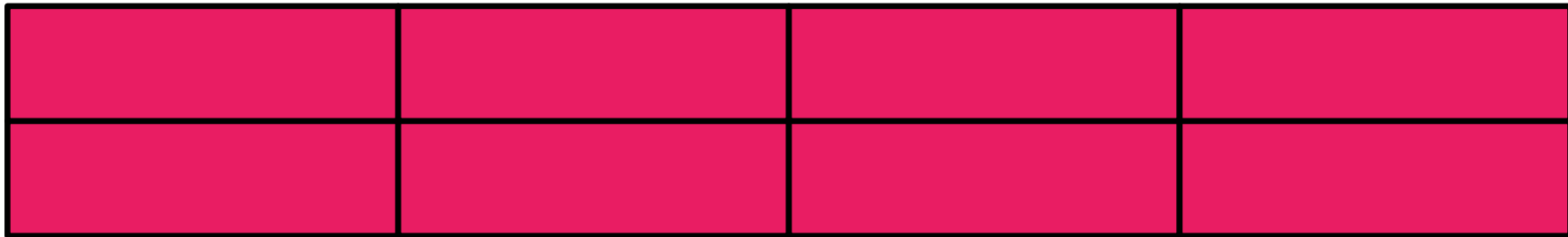
directories: reading direntries

— — —

```
dir_inode = {  
    type: 'd'  
    owner: "username"  
    size: 1  
    blocks: [4]  
}
```

```
struct fs_direntry {  
    char    name[FS_MAXFILENAME + 1];  
    uint32_t inode_block;  
};
```

```
fs_direntry entries[FS_DIRENTRIES];
```

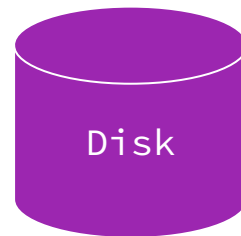


directories: reading direntries

```
dir_inode = {  
    type: 'd'  
    owner: "username"  
    size: 1  
    blocks: [4]  
}
```

```
struct fs_direntry {  
    char    name[FS_MAXFILENAME + 1];  
    uint32_t inode_block;  
};
```

```
fs_direntry entries[FS_DIRENTRIES];  
disk_readblock(entries_block, entries);
```



p 4 . c p p 0 ... 2	p 3 . c p p 0 ... 6	x x x x x x x ... 0	...

directories: reading direntries

```
dir_inode = {  
    type: 'd'  
    owner: "username"  
    size: 1  
    blocks: [4]  
}
```

```
struct fs_direntry {  
    char    name[FS_MAXFILENAME + 1];  
    uint32_t inode_block;  
};
```

```
cout << entries[0].name;
```

"p4.cpp"

p 4 . c p p 0 ... 2

p 3 . c p p 0 ... 6

x x x x x x x ... 0

...

directories: reading direntries

```
dir_inode = {  
    type: 'd'  
    owner: "username"  
    size: 1  
    blocks: [4]  
}
```

```
struct fs_direntry {  
    char    name[FS_MAXFILENAME + 1];  
    uint32_t inode_block;  
};
```

```
cout << entries[1].name;
```

"p3.cpp"

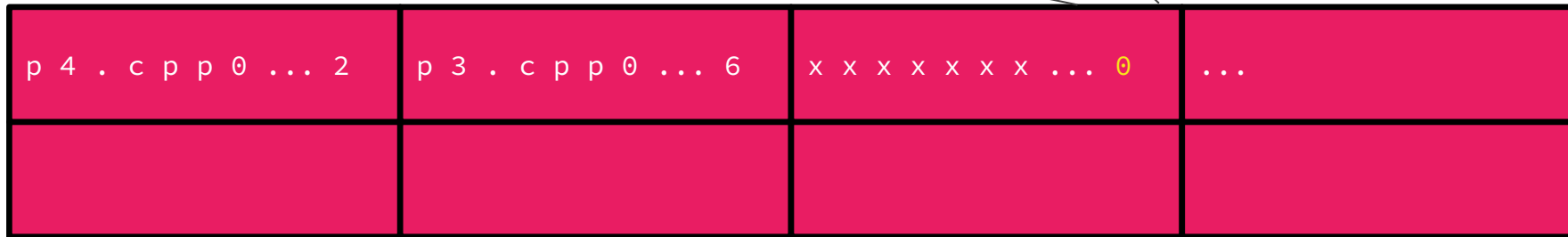
p 4 . c p p 0 ... 2	p 3 . c p p 0 ... 6	x x x x x x x ... 0	...

directories: reading direntries

```
dir_inode = {  
    type: 'd'  
    owner: "username"  
    size: 1  
    blocks: [4]  
}
```

```
struct fs_direntry {  
    char    name[FS_MAXFILENAME + 1];  
    uint32_t inode_block;  
};
```

```
cout << entries[2].inode_block;
```

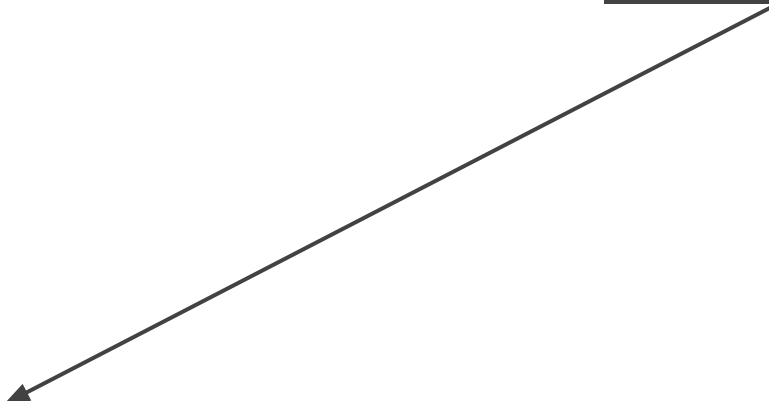


p 4 . c p p 0 ... 2	p 3 . c p p 0 ... 6	x x x x x x x ... 0	...

directories: adding a direntry

```
dir_inode = {  
  type: 'd'  
  owner: "username"  
  size: 1  
  blocks: [4]  
}
```

```
notes_inode = {  
  type: 'f'  
  owner: "username"  
  size: 2  
  blocks: [11, 12]  
}
```



p 4 . c p p 0 ... 2	p 3 . c p p 0 ... 6	x x x x x x x ... 0	...

directories: adding a direntry

```
dir_inode = {  
  type: 'd'  
  owner: "username"  
  size: 1  
  blocks: [4]  
}
```

```
notes_inode = {  
  type: 'f'  
  owner: "username"  
  size: 2  
  blocks: [11, 12]  
}
```

p 4 . c p p 0 ... 2	p 3 . c p p 0 ... 6	notes 0 0 ... 9	...

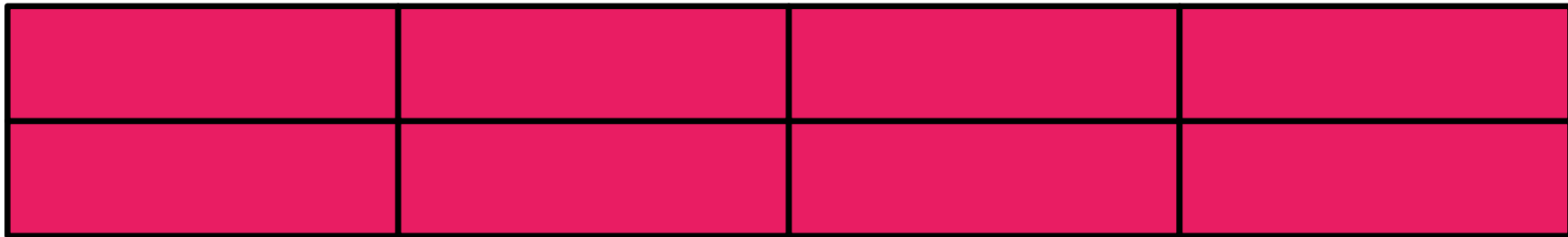
directories: adding a direntry

— — —

```
dir_inode = {  
    type: 'd'  
    owner: "username"  
    size: 1  
    blocks: [4]  
}
```

```
struct fs_dirent {  
    char    name[FS_MAXFILENAME + 1];  
    uint32_t inode_block;  
};
```

```
fs_dirent entries[FS_DIRENTRIES];
```

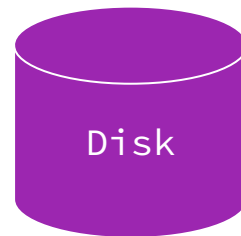


directories: adding a direntry

```
dir_inode = {  
    type: 'd'  
    owner: "username"  
    size: 1  
    blocks: [4]  
}
```

```
struct fs_direntry {  
    char    name[FS_MAXFILENAME + 1];  
    uint32_t inode_block;  
};
```

```
fs_direntry entries[FS_DIRENTRIES];  
disk_readblock(entries_block, entries);
```



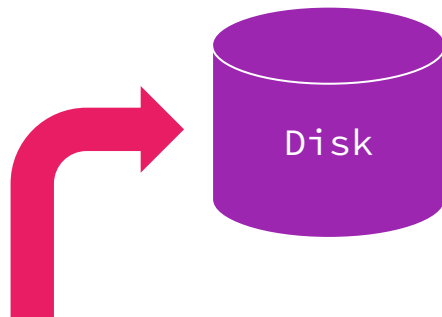
p 4 . c p p 0 ... 2	p 3 . c p p 0 ... 6	x x x x x x x ... 0	...

directories: adding a direntry

```
dir_inode = {  
    type: 'd'  
    owner: "username"  
    size: 1  
    blocks: [4]  
}
```

```
struct fs_dirent {  
    char    name[FS_MAXFILENAME + 1];  
    uint32_t inode_block;  
};
```

```
fs_dirent entries[FS_DIRENTRIES];  
disk_readblock(entries_block, entries);  
...
```



p 4 . c p p 0 ... 2	p 3 . c p p 0 ... 6	n o t e s 0 0 ... 9	...

File Systems Exercise

File Systems

Use the `fs_inode` and `fs_dirent` data structures from Project 4.
Note: Blocks are 512 bytes, and inodes are block-sized.

A. What is the minimum and maximum number of disk blocks that must be brought into memory in order to read the first block of the file `/etc/passwd`?

File Systems

B. A big file `/bigfile` that is 4 blocks in size is stored in the file system.

1. Draw out how `/bigfile` is laid out on disk. Include all inodes and data blocks required to store the file. Assume there are no other files.
2. A user program makes a read request from offset 1 from `/bigfile`. How many disk accesses are required to process the request?
3. How many disk accesses are required to process a read request on offset 2 of `/bigfile`? How many accesses would it be if we had cached the blocks from part b?

File Systems

B. cont.

4. If we wanted to add a new directory, /foo, how many new disk blocks do we need to allocate? Update your diagram from part a with the new disk blocks.

5. If we wanted to add a new file, /foo/bar, how many new disk blocks do we need to allocate? Update your diagram from part A with the new disk blocks.

Installing Boost

Boost

- C++ Library
- Provides more powerful multithreading utilities than STL
 - reader/writer locks
- Section 5.3 and 10 of P4 Spec

Install Boost in lab today!

- Lookup instruction for your operating system

Try compiling simple program with `<boost/thread.hpp>`

Boost Install Commands

— — —

macOS:

```
brew install boost
```

Ubuntu:

```
sudo apt update
```

```
sudo apt upgrade
```

```
sudo apt-get install libboost-all-dev
```

These alone might not export boost to your path.

Boost Path Specification (Not Using CMake)

```
export CPLUS_INCLUDE_PATH=<your_boost_path_include>:
```

```
export LIBRARY_PATH=<your_boost_path_lib>:
```

```
export LD_LIBRARY_PATH=<your_boost_path_lib>:
```

E.g.: On Mac with Brew:

```
brew info boost to find your boost installation path.
```

```
haytham918@:15:07:~:$ brew info boost
==> boost: stable 1.86.0 (bottled), HEAD
Collection of portable C++ source libraries
https://www.boost.org/
Installed
/opt/homebrew/Cellar/boost/1.84.0_1 (16,054 files, 498.2MB) *
  Poured from bottle using the formulae.brew.sh API on 2024-04-07 at 21:49:36
From: https://github.com/Homebrew/homebrew-core/blob/HEAD/Formula/b/boost.rb
License: BSL-1.0
==> Dependencies
Required: icu4c@76 ✓, xz ✓, zstd ✓
==> Options
--HEAD
      Install HEAD version
```

```
haytham918@:15:14:~:$ export CPLUS_INCLUDE_PATH=/opt/homebrew/Cellar/boost/1.84.0_1/include: && \
export LIBRARY_PATH=/opt/homebrew/Cellar/boost/1.84.0_1/lib: && \
export LD_LIBRARY_PATH=/opt/homebrew/Cellar/boost/1.84.0_1/lib: □
```

Test Boost

Compile and Run the following simple program (use the Makefile provided for p4)

```
1  #include <boost/thread.hpp>
2  #include <iostream>
3
4  boost::mutex my_mutex;
5
6  int main(){
7      my_mutex.lock();
8      std::cout << "LOCKED\n";
9      my_mutex.unlock();
10     std::cout << "WORKING!\n";
11 }
```

Questions?

CMake Tips

CMake with Boost

- Makefiles are not used for large projects in the real world
- CMake builds your Makefile for you!
 - Automatically finds your Boost install
- We provide you one.
 - Install CMake with brew or apt-get

Workflow:

```
mkdir build
```

```
cd build
```

```
cmake ..
```

```
make -j
```

CMake with Boost (cont.)

— — —

```
1 cmake_minimum_required(VERSION 3.16)
2
3 project(E ECS482_P4)
4 set(CMAKE_CXX_STANDARD 20)
5
6 add_compile_definitions(DEBUG)
7 # Compiler Flags
8 list(APPEND CMAKE_CXX_FLAGS "-g -O0 -Wall -Wextra -Wpedantic")
9
10 find_package(Boost REQUIRED thread system)
11
12 add_library(libfs_client STATIC IMPORTED)
13 add_library(libfs_server STATIC IMPORTED)
14
15 if (APPLE)
16     add_compile_options(-D_XOPEN_SOURCE)
17     set_target_properties(libfs_client PROPERTIES IMPORTED_LOCATION ${CMAKE_CURRENT_LIST_DIR}/external/libfs_client_macos.o)
18     set_target_properties(libfs_server PROPERTIES IMPORTED_LOCATION ${CMAKE_CURRENT_LIST_DIR}/external/libfs_server_macos.o)
19 else ()
20     set_target_properties(libfs_client PROPERTIES IMPORTED_LOCATION ${CMAKE_CURRENT_LIST_DIR}/external/libfs_client.o)
21     set_target_properties(libfs_server PROPERTIES IMPORTED_LOCATION ${CMAKE_CURRENT_LIST_DIR}/external/libfs_server.o)
22 endif ()
23
24 file(GLOB SRC_FILES
25     "src/*.h"
26     "src/*.cpp"
27 )
28
29 add_executable(fs ${SRC_FILES})
30 target_link_libraries(fs libfs_server ${CMAKE_DL_LIBS} Boost::thread)
31 add_custom_command(PRE_BUILD TARGET fs COMMAND ${CMAKE_CURRENT_LIST_DIR}/autotag.sh push)
32
33 add_subdirectory(tests)
34
```